DATA WAREHOUSE

Durante el curso construimos el flujo de un data warehouse ejecutado localmente a partir del repositorio DE-DW.

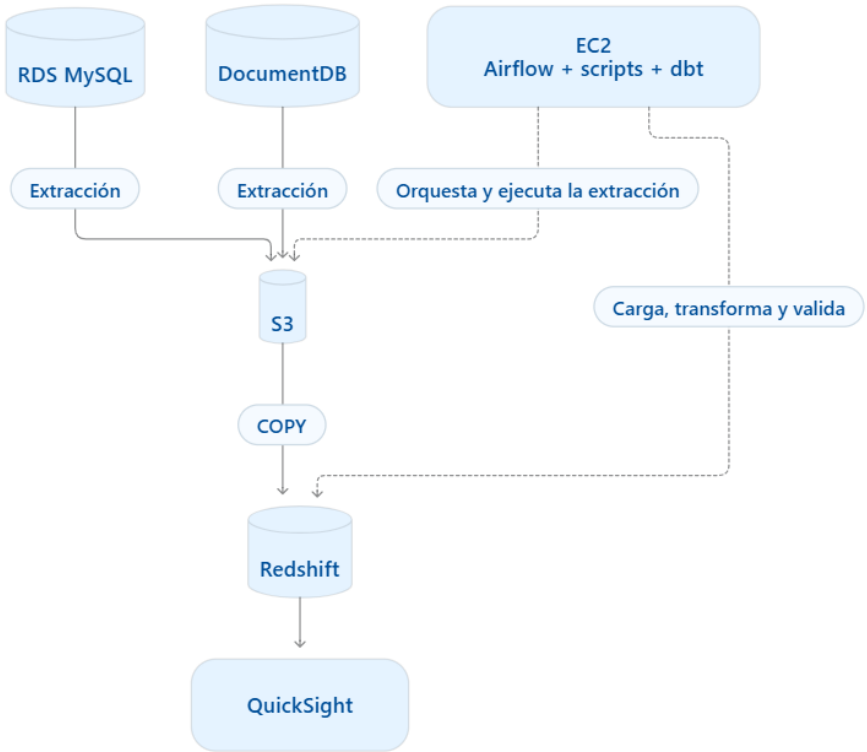
El siguiente reto consiste en migrar esta solución a AWS, conservando sus reglas de negocio y garantizando que el procesamiento sea reproducible, verificable y seguro.

La solución debe integrar fuentes relacionales y documentales, construir un data warehouse en Amazon Redshift y presentar sus resultados mediante una visualización en Amazon QuickSight.

1. Objetivo

- Implementar y demostrar un pipeline de datos de extremo a extremo en AWS que permita:
- Extraer información desde Amazon RDS y Amazon DocumentDB.
- Conservar los datos extraídos en Amazon S3.
- Cargar y transformar los datos en Amazon Redshift.
- Ejecutar Apache Airflow y dbt en una instancia Amazon EC2.
- Publicar un dashboard sencillo en Amazon QuickSight.
- Verificar calidad, cargas incrementales, idempotencia e historia de cambios.

2. Arquitectura



3. Requisitos funcionales

3.1. Fuentes de datos

1. Migrar la fuente MySQL local a Amazon RDS for MySQL.
2. Migrar MongoDB local a Amazon DocumentDB.
3. Adaptar los scripts de generación y conexión.
4. Se pueden seguir usando los mis datos que se emplearon en local
5. Documentar los ajustes necesarios por diferencias entre MongoDB y DocumentDB.

3.2. Extracción y almacenamiento en S3

Implementar una extracción inicial y una extracción incremental por lote.

Cada extracción debe:

- Escribir los datos en un bucket S3 privado.
- Identificar la fuente, entidad y lote en la ruta de los objetos.
- Registrar fecha de extracción, identificador del lote y cantidad de registros.
- Conservar los archivos necesarios para reprocesar un lote sin consultar nuevamente las fuentes.
- Evitar sobrescribir silenciosamente los datos de otros lotes.

Ejemplo de organización:

```
s3://<bucket>/raw/<fuente>/<entidad>/batch_id=<id>/
```

Se permite utilizar CSV, JSON o Parquet, justificando la elección y su compatibilidad con la carga a Redshift.

3.3. Data warehouse en Redshift

Sustituir DuckDB por Amazon Redshift.

Se permite utilizar Redshift Serverless o un clúster aprovisionado, según los recursos disponibles en la cuenta del curso.

La implementación debe:

1. Cargar los archivos desde S3 mediante `COPY`.
2. Utilizar un rol IAM para autorizar el acceso de Redshift a S3.

3. Mantener las responsabilidades de las capas `raw`, `intermediate`, `quarantine` y `marts`.
4. Conservar el modelo dimensional y los procesos de negocio del proyecto.
5. Implementar historia de cambios SCD tipo 2.
6. Adaptar tipos de datos, funciones SQL y configuraciones de dbt a Redshift.
7. Registrar los rechazos junto con su motivo y lote de origen.

3.4. Orquestación con Airflow

Construir o adaptar un DAG que coordine:

1. Validación de conexiones y parámetros.
2. Extracción de las fuentes.
3. Escritura de archivos en S3.
4. Carga a Redshift.
5. Transformaciones y snapshots de dbt.
6. Pruebas de calidad.
7. Registro del resultado del procesamiento.

Airflow y dbt deben ejecutarse en la misma instancia EC2. Las transformaciones SQL enviadas por dbt deben ejecutarse en Redshift.

El flujo debe incluir dependencias explícitas, reintentos acotados y logs que permitan identificar el lote y la tarea que fallaron.

El pipeline debe poder ejecutarse desde Airflow sin intervención manual entre etapas.

3.5. Calidad, incrementalidad y recuperación

Presentar evidencia de los siguientes escenarios:

Escenario	Resultado esperado
Carga inicial	Datos disponibles en los modelos analíticos
Registros defectuosos	Rechazos identificados y trazables
Repetición de un lote exitoso	Sin duplicación de hechos ni versiones históricas
Nuevo lote incremental	Incorporación correcta de nuevos registros
Cambio en un atributo historizado	Nueva versión SCD tipo 2 y cierre de la anterior
Fallo controlado y reejecución	Recuperación sin pérdida ni duplicación

Incluir pruebas de unicidad, campos obligatorios, relaciones y reglas de negocio. Conciliar al menos un indicador agregado entre los datos válidos de origen y el modelo analítico.

Documentar cómo se identifica un lote procesado y en qué momento se actualiza su estado o marca de avance.

3.6. Consumo con QuickSight

Amazon QuickSight es un servicio de AWS que permite crear gráficas y dashboards para explorar y presentar datos. En este proyecto, se conectará a Redshift para visualizar la información del data warehouse, como las ventas por fecha o categoría, y aplicar filtros sin tener que escribir consultas SQL cada vez.

Así que se debe crear un dashboard con al menos:

- Una gráfica de ventas por fecha, categoría u otra dimensión del modelo.
- Un indicador de ventas totales.
- Un filtro relevante para explorar los resultados.

El dashboard debe consumir una tabla o vista de `marts` en Redshift.

Entregar la consulta SQL utilizada para validar el indicador y demostrar que coincide con el valor presentado, considerando los mismos filtros y la misma fecha de actualización.

Si se utiliza SPICE, documentar cómo se actualiza el dataset después de procesar un nuevo lote.

4. Entregables

1. **Repositorio de código**
   - DAGs de Airflow y scripts de extracción y carga.
   - Proyecto dbt adaptado a Redshift.
   - Configuración de despliegue sin secretos.
   - Archivo de ejemplo con las variables requeridas.
2. **Documentación**
   - Diagrama de arquitectura.
   - Instrucciones reproducibles de despliegue, ejecución y eliminación de recursos.
   - Decisiones técnicas y diferencias frente a la versión local.
   - Estimación de costos y controles de seguridad implementados.
3. **Evidencias de funcionamiento**
   - Ejecución exitosa del pipeline en Airflow.
   - Resultados de las pruebas de dbt.
   - Validación de carga inicial, calidad, incrementalidad, idempotencia, SCD tipo 2 y recuperación ante fallos.
   - Dashboard de QuickSight y consulta SQL que permita verificar el indicador presentado.

4. **Demostración**
    - Procesar un nuevo lote desde las fuentes hasta el dashboard.
    - Explicar cómo se identifica y recupera una ejecución fallida.

Las capturas complementan la entrega, pero no sustituyen el código ni las instrucciones reproducibles.

Ingeniería de Datos
Arquitecturas de Datos
2026-2
DATA LAKE

En este proyecto construiremos un data lake para recopilar, almacenar y analizar noticias provenientes de fuentes RSS.

La solución utilizará una arquitectura medallón en Amazon S3, con capas Bronze, Silver y Gold. Apache Airflow ejecutado en Amazon EC2 orquestará el procesamiento, Amazon Comprehend enriquecerá los textos y Amazon Athena permitirá consultar los resultados mediante SQL.

AWS Glue Data Catalog almacenará los metadatos de las tablas disponibles para consulta.

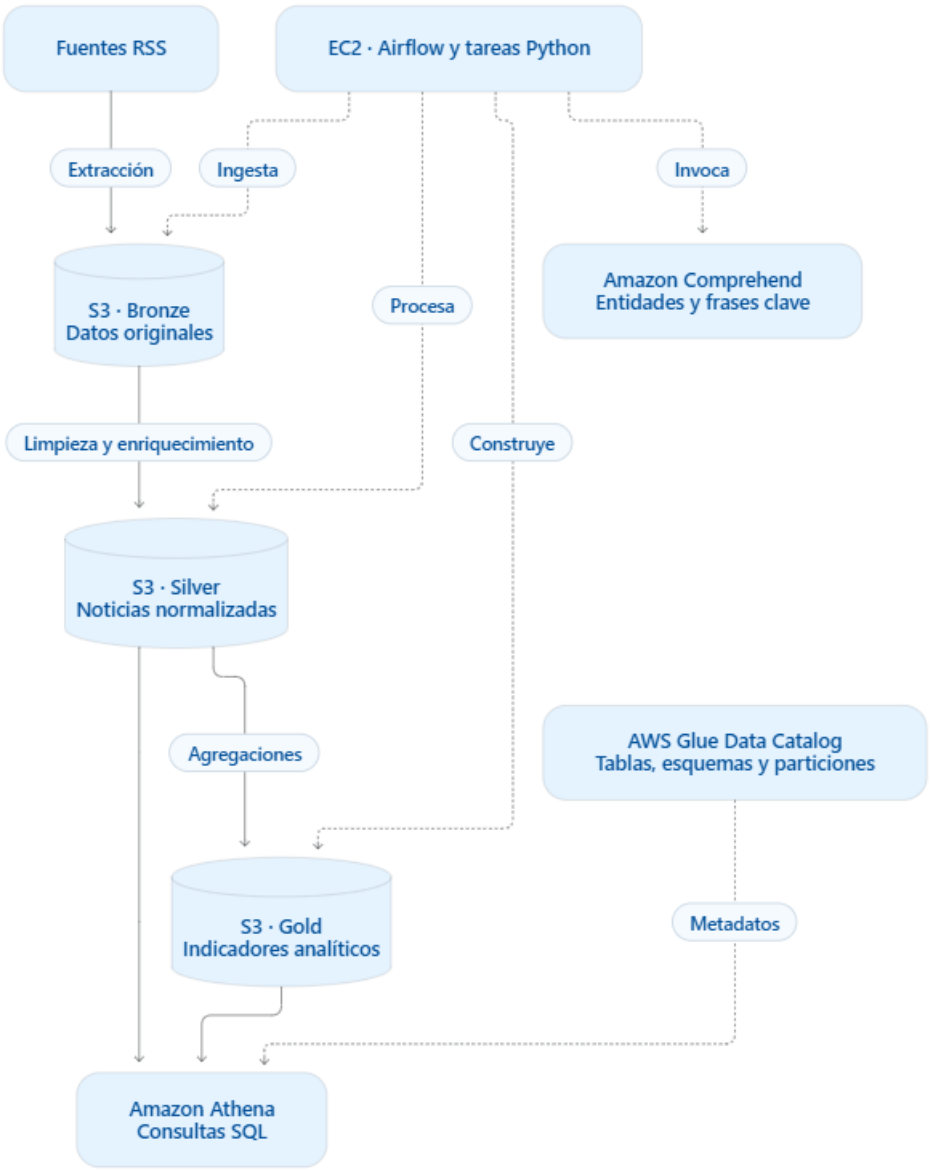
Todas las zonas del data lake deben crear a partir de un solo bucket.

1. Objetivo

Implementar un pipeline de datos de extremo a extremo que permita:

- Extraer noticias desde diferentes feeds RSS.
- Conservar los datos originales en S3.
- Limpiar, validar y deduplicar las noticias.
- Extraer entidades y frases clave con Amazon Comprehend.
- Construir conjuntos de datos analíticos mediante una arquitectura medallón.
- Registrar las tablas en AWS Glue Data Catalog.
- Consultar los datos con Amazon Athena.
- Demostrar calidad, incrementalidad, idempotencia y recuperación ante fallos.

2. Arquitectura



4. Requisitos funcionales

4.1. Fuentes RSS

Seleccionar tres secciones del directorio RSS de El Tiempo y utilizar el feed RSS correspondiente a cada una como fuente de datos.

La implementación debe:

1. Identificar las URL de los feeds. La página del directorio no corresponde a un feed.
2. Mantener una configuración con el medio, la sección y la URL de cada fuente.
3. Extraer los campos disponibles: identificador, título, enlace, descripción y fecha de publicación.
4. Registrar la fuente y la fecha de extracción.
5. Manejar respuestas vacías, errores HTTP y XML malformado.

Se utilizará únicamente el contenido disponible en los RSS. No se requiere descargar el texto completo de las páginas enlazadas.

4.2. Ingesta y capa Bronze

Implementar tareas Python, ejecutadas desde Airflow en EC2, que consulten los feeds y almacenen en S3 el contenido original de las respuestas válidas.

Cada lote debe registrar:

- Identificador de ejecución.
- Fuente y URL consultada.
- Fecha y hora de ingesta en UTC.
- Estado HTTP.
- Hash del contenido.
- Ubicación del archivo original en S3.

Ejemplo de organización:

bronze/source=<fuente>/ingestion_date=AAAA-MM-DD/run_id=<id>/

Bronze debe conservar los datos originales y permitir reprocesar un lote sin consultar nuevamente las fuentes.

Distinguir entre una nueva extracción y el reprocesamiento de un lote existente. Reprocesar un lote no debe crear otra copia lógica del mismo lote.

4.3. Capa Silver: limpieza y enriquecimiento

Transformar los archivos de Bronze en conjuntos de datos estructurados.

El procesamiento debe:

1. Interpretar el XML y extraer las noticias.
2. Limpiar etiquetas HTML del título y la descripción.
3. Normalizar fechas a UTC, conservando el valor original.
4. Validar los campos definidos como obligatorios.
5. Detectar noticias repetidas dentro de un lote y entre lotes.
6. Identificar cambios en noticias previamente procesadas.
7. Conservar la referencia al archivo y lote de Bronze.
8. Separar los registros inválidos en una ruta de cuarentena con su motivo.

Definir una estrategia de identificación basada en GUID, enlace u otros campos disponibles. Documentar cómo se manejan las noticias presentes en varios feeds y las limitaciones de la estrategia.

Enriquecimiento con Amazon Comprehend

Amazon Comprehend es un servicio de AWS que utiliza procesamiento de lenguaje natural para analizar textos. En este proyecto, permitirá identificar personas, organizaciones, lugares y frases clave en los títulos y descripciones de las noticias, enriqueciendo los datos para consultarlos con Athena.

Analizar el título y la descripción disponibles para extraer:

- Personas.
- Organizaciones.
- Lugares.
- Frases clave.

Guardar el texto analizado, su hash, el idioma utilizado, la fecha de análisis y las puntuaciones de confianza devueltas por el servicio.

Evitar nuevas llamadas a Comprehend cuando el texto ya haya sido analizado y no haya cambiado.

Los textos vacíos, demasiado extensos o en idiomas no contemplados deben tener un tratamiento explícito y un estado de procesamiento.

Almacenar en Parquet, como mínimo:

Conjunto	Granularidad
news	Una fila por noticia vigente
news_entities	Una fila por entidad distinta detectada en cada noticia vigente
news_key_phrases	Una fila por frase clave distinta detectada en cada noticia vigente

Las entidades representan menciones detectadas automáticamente. No se exige resolver alias ni determinar si dos nombres corresponden a la misma persona.

4.4. Capa Gold y consulta con Athena

Construir conjuntos de datos agregados que permitan responder:

1. ¿Cuántas noticias se publican por día, medio y sección?
2. ¿Qué personas, organizaciones y lugares aparecen en más noticias?
3. ¿Qué frases clave son más frecuentes por sección?
4. ¿Cómo cambia la presencia de una entidad entre fechas?

Para medir la presencia de una entidad, contar noticias distintas y evitar que las repeticiones dentro de un texto inflen el indicador.

Documentar la granularidad de cada tabla, la definición de sus métricas y el tratamiento de noticias sin fecha de publicación válida.

Registro con AWS Glue

Registrar las tablas de Silver y Gold en **AWS Glue Data Catalog**, incluyendo:

- Ubicación de los archivos en S3.
- Columnas y tipos de datos.
- Formato de almacenamiento.
- Claves y valores de partición.

Se permite utilizar definiciones explícitas de tablas o AWS Glue Crawlers. Explicar cómo se actualizan los metadatos cuando aparecen nuevas particiones.

Un crawler no reemplaza las validaciones de calidad ni las transformaciones del pipeline.

Consumo con Athena

Entregar consultas SQL que permitan:

- Consultar noticias por fecha y sección.

- Obtener las diez organizaciones presentes en más noticias.
- Comparar la presencia de una entidad entre días.
- Consultar las frases clave más frecuentes por sección.
- Conciliar una agregación de Gold contra Silver.

Silver y Gold deben almacenarse en Parquet y utilizar una estrategia de particionamiento por fecha acorde con las consultas.

Comparar los bytes escaneados de una consulta con filtro de partición y otra sin ese filtro.

4.5. Calidad, incrementalidad y recuperación

Presentar evidencia de los siguientes escenarios:

Escenario	Resultado esperado
Carga inicial	Datos disponibles en Bronze, Silver y Gold, y consultables en Athena
Registros defectuosos	Rechazos identificados, con motivo y trazabilidad hasta Bronze
Repetición de un lote exitoso	Sin duplicados en Silver ni doble conteo en Gold
Nuevo lote incremental	Incorporación de noticias nuevas sin reprocesar innecesariamente las existentes
Noticia con contenido modificado	Actualización de la versión vigente y nuevo enriquecimiento únicamente cuando cambia el texto analizado
Fallo controlado y reejecución	Recuperación sin pérdida ni duplicación de resultados
Reprocesamiento desde Bronze	Reconstrucción de resultados sin volver a consultar los RSS
Nueva partición	Datos disponibles en Athena después de actualizar los metadatos correspondientes

Incluir validaciones de:

- Unicidad del identificador de noticia en el conjunto vigente.
- Presencia de campos obligatorios.
- Validez de fechas y enlaces.
- Ausencia de entidades o frases clave sin una noticia asociada.
- Conciliación de indicadores entre Silver y Gold.
- Identificación de registros nuevos, repetidos, actualizados y rechazados.

Registrar por lote los conteos de entrada y el resultado de su procesamiento. Explicar las diferencias entre registros recibidos y noticias únicas publicadas.

Para demostrar cambios en una noticia, se permite utilizar una copia controlada de un RSS almacenado en Bronze. No se debe depender de que un medio publique una actualización durante la demostración.

Documentar cómo se escriben y publican los archivos para que una reejecución no deje resultados duplicados o parciales visibles en Athena.

La conservación de versiones históricas de noticias es opcional. Si se implementa, debe distinguirse claramente la versión vigente y evitar que las versiones anteriores inflen las métricas.

4.6. Orquestación con Airflow en EC2

Desplegar Apache Airflow en una instancia EC2. Se permite utilizar Docker Compose.

Construir un DAG que coordine:

1. Validación de configuración y acceso a los servicios.
2. Extracción de los feeds y escritura en Bronze.
3. Limpieza, validación y deduplicación.
4. Enriquecimiento con Comprehend.
5. Publicación de Silver.
6. Construcción de Gold.
7. Actualización de tablas o particiones en Glue Data Catalog.
8. Validaciones finales con Athena.
9. Registro del estado del procesamiento.

El DAG debe incluir:

- Ejecución programada durante el laboratorio.
- Dependencias explícitas.
- Reintentos acotados para fallos transitorios.
- Logs con fuente, lote y tarea.
- Control de concurrencia para evitar escrituras conflictivas.
- Parámetros para reprocesar un lote desde Bronze.

Si una fuente falla, se debe identificar si la ejecución fue completa, parcial o fallida. Una fuente no disponible no debe interpretarse silenciosamente como una fuente sin noticias.

El pipeline debe ejecutarse sin intervención manual entre etapas.

5. Entregables

1. **Repositorio de código**
   - DAGs de Airflow.
   - Scripts de ingesta, transformación y enriquecimiento.
   - Definiciones de tablas o configuración de crawlers.
   - Consultas SQL de Athena.
   - Configuración de despliegue y variables de ejemplo sin secretos.
2. **Documentación**
   - Diagrama de arquitectura.
   - Fuentes RSS seleccionadas.
   - Diccionario de datos de Silver y Gold.
   - Reglas de calidad, identificación y actualización.
   - Estrategia de incrementalidad y recuperación.
   - Instrucciones reproducibles de despliegue, ejecución y eliminación.
   - Estimación de costos.
3. **Evidencias**
   - Ejecución del DAG en Airflow.
   - Archivos de Bronze, Silver y Gold.
   - Resultados de Comprehend.
   - Tablas y particiones en Glue Data Catalog.
   - Consultas y resultados en Athena.
   - Validación de los escenarios de calidad, incrementalidad y recuperación.
4. **Demostración**
   - Ejecutar una ingesta completa.
   - Consultar los indicadores en Athena.
   - Reprocesar un lote sin duplicar resultados.
   - Recuperar una ejecución después de un fallo controlado.
   - Explicar una limitación del enriquecimiento de texto.

Las capturas complementan la entrega, pero no sustituyen el código ni las instrucciones reproducibles.